Final Project:
What's the Number? A CNN Accelerator Simulator in C++
Appendix: How to Use Header File

1. What is a Header File?

A header file (.h / .hpp) is like a "tool catalog" or "user manual." It doesn't tell you how the tools work internally, but it clearly lists all the available names, functions, and the required inputs and outputs you can use.

In contrast, the source file (.cpp) is like the "factory" where those tools are built. It contains the detailed implementation of the functions and classes that were only declared in the header file.

In other words, the header file tells you what functions exist, while the source file shows you how they work.

The roles of .h, .cpp, and main.cpp files are as follows:

File	Student Permission	Key Concept
hardware_architecture.h	Reference only	Lists all available functions and classes.
hardware_architecture.cpp	Reference only	Contains the actual computation process, **do not modify**.
main.cpp	**You write this part**	Include hardware_architecture.cpp and call the provided functions here.

2. How to Read Header File?

The <hardware_architecture.h> file tells you which tools (functions) are available. It only specifies the return value type and input parameters, while the corresponding hardware_architecture.cpp file describes the actual functionality and behavior of each function.

For example:

In <hardware_architecture.h>, you can see that output() is a function that returns no value (void) and takes one integer parameter (int x):

hardware_architecture.h

```
1  #ifndef HARDWARE_ARCHITECTURE_H
2  #define HARDWARE_ARCHITECTURE_H
3
4  void output(int number);
```

And in <hardware_architecture.cpp>, you can see the detailed implementation logic of the output() function:

```
13  void output(int number) {
14      cout << "number = " << number << endl;
15  }
```

3. How to Use Header File in Code::Block?

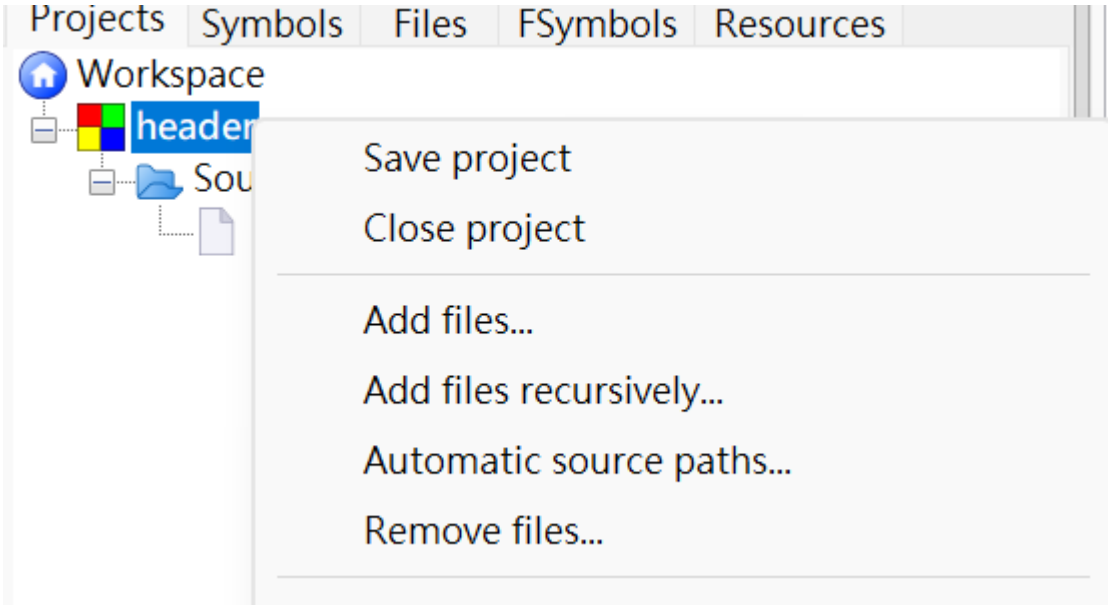
Step 1: Create a new C++ project.

Step 2: Place both hardware_architecture.h and hardware_architecture.cpp into the project folder.

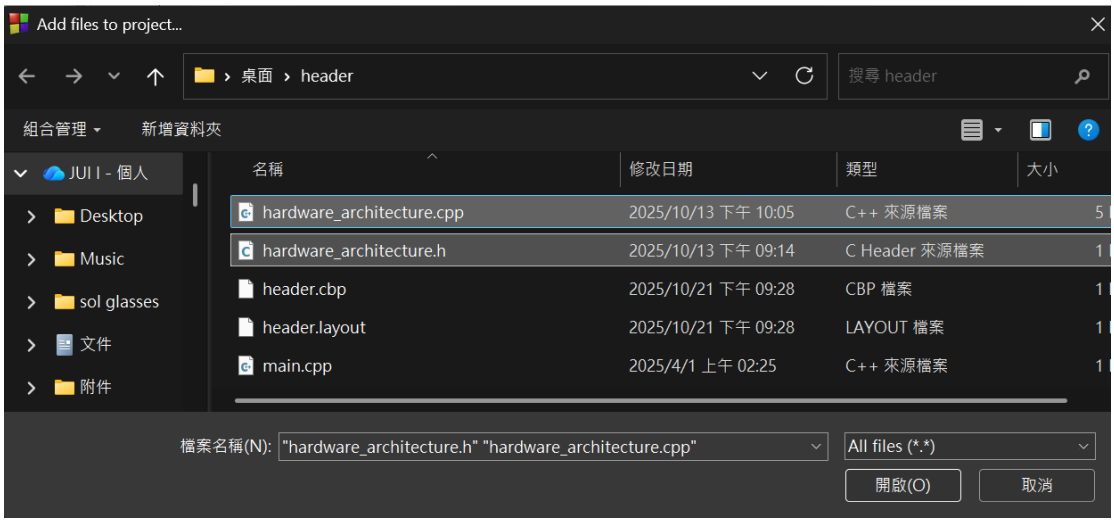


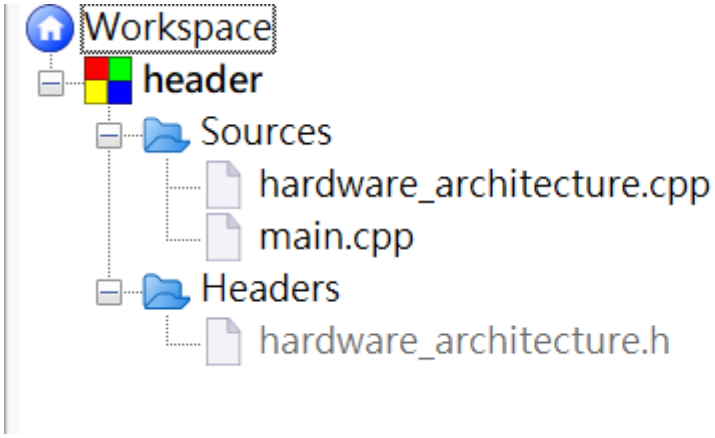
Step 3: In the window on the left side of Code::Blocks, right-click your project name.

Step 4: Choose **"Add Files."**

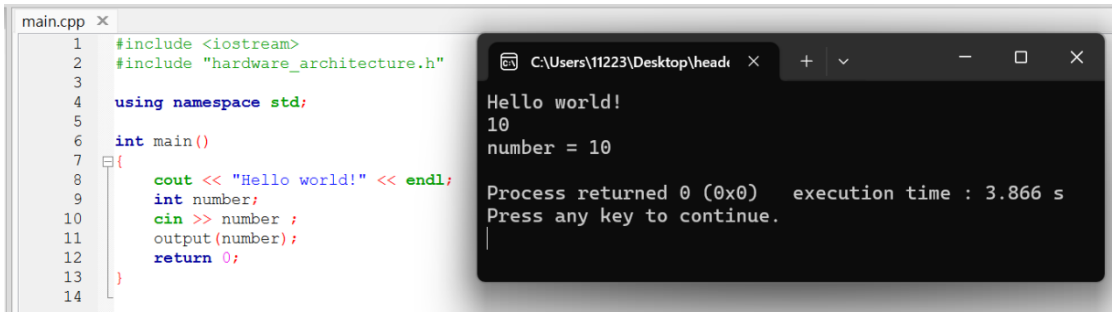


Step 5: Add both<hardware_architecture.h> and <hardware_architecture.cpp> to the project.





Step 6: Open main.cpp and include the header file at the beginning of the code.
You can then directly call any functions declared in the header file — for example, output().
After compiling and running the project, the program will execute the behavior of output() as defined in <hardware_architecture.cpp>.



4. Functions of hardware_architecture

Function	Description
void output(int number)	Print the given number to the console, used to verify header file usage. Not allowed to be called from main.cpp.
float return_energy()	Return the current value of the energy_counter. Callable in main.cpp.
int return_step()	Return the current value of the step_counter. Callable in main.cpp.
void reset_buffer_and_matrix()	Reset all line buffers and matrices (input_matrix, psum_matrix, weight_matrix, output_matrix). Callable in main.cpp.
void update_line_buffer(float buffer_1_new, float buffer_2_new)	Update both line buffers by shifting old data and inserting a new values at the front.
float get_item_from_line_buffer (int item, int buffer_number)	Retrieve a specific element from the chosen line buffer (buffer 1 or buffer 2).
float* input_selection(bool pe1_use_buffer, int lb_1_index, bool pe2_use_buffer, int lb_2_index, float new_data[3])	Update the input matrix for PE1, PE4, and PE7 using either line buffer data or new external data and update the energy counter accordingly. Calls pe_array_computation() internally. Callable in main.cpp.
void update_matrixs(float* pe_output_matrix)	Update the psum_matrix and input matrix based on PE computation output and shift line buffers.
void set_weight(float (*weight_input)[3])	Load a new 3×3 weight matrix into the system and update the energy counter. Callable from main.cpp.
float* pe_array_computation()	Instruct the 3×3 PE array to perform convolution computation, updating the output and internal matrices, and incrementing the step counter.